Technical Recreation Guide for the Hybrid FCS–MCV EV Charging Experiments

SEEMS Agentic AI-based Hybrid EV Charging Simulator
Joy Dutta

April 2026

Purpose and Scope

This document is a technical specification of the normal and extended evaluation experiments used in the IEEE Network Magazine paper "Agentic AI for Dynamic EV Charging". It consolidates the scenario assumptions, controller modes, mathematical formulas, queueing logic, MCV deployment rules, LLM boundedness checks, output artifacts, and reported results.

The accompanying implementation (the `ev_charging_agents` folder) will be released online after the paper is accepted. Until then, this document is intended to make the experimental design and reported results transparent and reviewable, and to enable later third-party reproduction once the code is publicly available.

The experiment supports the proof-of-concept subsection and Table II of the paper *Agentic AI for Dynamic EV Charging: Explainable Coordination of Fixed Charging Stations and Mobile Charging Vehicles*. The article has limited space, so this guide records the implementation details that are intentionally omitted from the magazine text: exact parameters, equations, controller modes, bounded LLM contracts, output files, and reproducible result summaries.

1 Relevant Implementation Files

File	Role
`ev_charging_agents/multi_fleet_simulation.py`	Core simulator. Defines the physical topology, EV request generation, FCS and MCV queue schedules, feasible-option generation, deterministic scoring, satisfaction, regret, LLM hooks, logs, and normal summary files.
`ev_charging_agents/extended_evaluation.py`	Extended experiment package. Adds controller modes, demand sweeps, threshold discovery, representative reruns, role ablations, compliance audit rows, and publication-oriented CSV/TeX outputs.
`ev_charging_agents/llm_brain.py`	Bounded OpenAI LLM interface. Contains the operator decision call and EV decision call. Both request JSON output and both are constrained by simulator generated data.
`ev_charging_agents/comparison_runner.py`	Normal deterministic-vs-LLM comparison runner for the 10-EV scenario. Matches EV IDs across runs and writes comparison text/CSV files.
`ev_charging_agents/operator_intent.py`	Operator policy mode and policy knobs. The reported experiments use `PolicyMode.REVENUE`.

File	Role
`run_deterministic.py`	Root-level runner for deterministic normal evaluation only.
`run_llm.py`	Root-level runner for LLM-assisted normal evaluation only.
`run_compare.py`	Root-level runner for normal deterministic-vs-LLM comparison.
`run_extended_eval.py` and `ev_charging_agents/run_extended_eval.py`	Runners for the extended evaluation. The second one is placed inside the folder for easy execution from Cursor when only that folder is visible.

2 Global Simulation Parameters

Symbol / field	Value
`RANDOM_SEED`	20260427
`AREA_SIZE_KM`	20.0
`CITY_SPEED_KMPH`	30.0
`SIMULATION_MINUTES`	10
`IDEAL_WAIT_TIME_MIN`	5.0
`TARGET_WAIT_TIME_MIN`	10.0
`DEPOT_LOCATION`	$(10, 10)$ km
`FCS_PORTS`	4
`FCS_POWER_PER_PORT_KW`	180.0
`MCV_COUNT`	6
`MCV_PORTS`	2
`MCV_POWER_PER_PORT_KW`	125.0
`MCV_BATTERY_CAPACITY_KWH`	300.0
`MCV_MIN_RESERVE_KWH`	20.0
`MCV_DEPOT_RECHARGE_KW`	180.0
`FCS_BASE_PRICE_USD_PER_KWH`	0.42
`MCV_BASE_PRICE_USD_PER_KWH`	0.50

3 Physical Topology

The region is a Manhattan grid. Coordinates are in km.

Resource	Location	Ports	Power per port
FCS_1	$(5, 5)$	4	180 kW
FCS_2	$(15, 5)$	4	180 kW
FCS_3	$(10, 15)$	4	180 kW
Depot	$(10, 10)$	N/A	N/A
MCV_1–MCV_6	Start at depot	2 each	125 kW each

3.1 Initial Peak-Hour Load

Each FCS uses a min-heap of port availability times. To simulate peak hour, the simulator seeds each FCS with active and queued service loads at time zero:

Station	Active remaining times (min)	Queued service durations (min)
FCS_1	16, 20, 24, 28	22, 25, 28
FCS_2	8, 12, 16, 20	18
FCS_3	12, 18, 24, 30	24, 26

The queued service durations are inserted into the same port schedule using `_reserve_without_record`. Therefore the queue state is represented by future port availability times, not by a separate queue list.

4 EV Request Factory

4.1 Base 10-EV Request Pattern

The base normal experiment uses 10 EVs arriving one per minute:

$$t_i = i - 1, \quad i \in \{1, \ldots, 10\}.$$

The base intent pattern is:

$$[\texttt{time}, \texttt{time}, \texttt{cost}, \texttt{cost}, \texttt{greedy}, \texttt{greedy}, \texttt{greedy}, \texttt{greedy}, \texttt{greedy}, \texttt{greedy}].$$

Each EV receives:

$$x_i, y_i \sim U(1, 19), \tag{1}$$

$$\text{current\_soc}_i \sim U(0.14, 0.38), \tag{2}$$

$$\text{target\_soc}_i \sim U(0.72, 0.84). \tag{3}$$

The random generator uses seed 20260427 for the base run unless overridden.

4.2 Vehicle Model Mix

Brand	Model	Usable kWh	Max DC kW	Avg. fast DC kW
Tesla	Model Y Long Range AWD	75.0	250.0	124.0
Tesla	Model 3 Long Range AWD	75.0	250.0	125.0
Tesla	Model Y Long Range AWD	75.0	250.0	124.0
Tesla	Model 3 Long Range AWD	75.0	250.0	125.0
Tesla	Model S Long Range	95.0	250.0	150.0
BYD	Seal 82.5 kWh RWD Design	82.5	150.0	100.0
BYD	ATTO 3	60.5	89.0	71.0
BYD	Seal 82.5 kWh RWD Design	82.5	150.0	100.0
Geely	EX5	60.2	160.0	95.0
Volkswagen	ID.4 Pro	77.0	175.0	120.0

These are representative model-class assumptions used to create realistic variation in service time. The simulator uses average fast-DC power rather than peak DC power when calculating charging duration.

4.3 Intent Preference Profiles

Intent	Price sens.	Wait tol.	Detour tol.	MCV accept.	Reliability	Risk aversion
`time_optimized`	0.15	0.05	0.25	0.85	0.65	0.50
`cost_optimized`	0.95	0.70	0.65	0.55	0.45	0.35
`greedy`	0.35	0.25	0.45	0.70	0.55	0.45

5 Charging Option Generation

For every EV e and every feasible resource r, the simulator constructs a `DecisionOption`.

5.1 Distance and Travel Time

The road distance is Manhattan distance:

$$d(e, r) = |x_e - x_r| + |y_e - y_r|.$$

Travel time is:

$$T_{e,r}^{travel} = \frac{d(e, r)}{v} \times 60,$$

where $v = 30$ km/h.

5.2 Energy Need

$$E_e = \max\left(8,\ (\text{target\_soc}_e - \text{current\_soc}_e)B_e\right),$$

where B_e is the usable battery capacity of EV e in kWh. The 8 kWh minimum avoids unrealistically tiny charging sessions.

5.3 Effective Charging Power and Service Time

$$P_{e,r}^{eff} = \max\left(20,\ \min(P_r^{port}, P_e^{avgDC})\right).$$

$$T_{e,r}^{service} = \frac{E_e}{P_{e,r}^{eff}} \times 60.$$

This means a high-power FCS cannot charge a vehicle faster than the vehicle's average fast-charging capability, and a slow vehicle is not treated as if it can use the full station peak power.

5.4 Port Scheduling and Wait Time

Each FCS and MCV has a heap of port availability times. For a candidate option:

$$T_{e,r}^{arrival} = t_e + T_{e,r}^{travel}.$$

Let A_r be the earliest port availability time and R_r be the resource availability time. For deployed MCVs, R_r includes depot-to-FCS travel time.

$$T_{e,r}^{start} = \max(T_{e,r}^{arrival}, A_r, R_r).$$

$$T_{e,r}^{wait} = \max(0, T_{e,r}^{start} - T_{e,r}^{arrival}).$$

$$T_{e,r}^{end} = T_{e,r}^{start} + T_{e,r}^{service}.$$

$$T_{e,r}^{total} = T_{e,r}^{travel} + T_{e,r}^{wait} + T_{e,r}^{service}.$$

5.5 Cost

$$C_{e,r} = E_e \pi_r,$$

where π_r is the resource price in USD/kWh.

5.6 MCV Feasibility

An MCV option is feasible only if the MCV is assigned to an FCS and:

$$E_{mcv}^{remaining} - E_e \geq E_{mcv}^{reserve},$$

where $E_{mcv}^{reserve} = 20$ kWh.
After reservation, the MCV onboard energy is reduced by E_e.

6 Operator Policy and Pricing Logic

The reported experiments use:

$$\texttt{POLICY\_MODE} = \texttt{revenue}.$$

The active policy knobs in `operator_intent.py` are:

Knob	Value
`pricing_aggressiveness`	0.1
`congestion_response_sensitivity`	0.1
`driver_preference_emphasis`	0.2
`fairness_weight`	0.5
`reassignment_aggressiveness`	0.2
`mcv_deployment_aggressiveness`	0.2

6.1 Pricing

In `revenue` mode:

$$\pi_{FCS} = 0.42 + 0.05\alpha,$$

$$\pi_{MCV} = 0.50 + 0.05\alpha,$$

where α is `pricing_aggressiveness`. Values are rounded to two decimal places.
In `peak_resilience` mode:

$$\pi_{FCS} = 0.42 + 0.02\alpha,$$

$$\pi_{MCV} = \max(0.38,\ 0.50 - 0.10\alpha).$$

The extended evaluation reported in the paper keeps revenue mode unchanged.

7 MCV Deployment Logic

The operator is called once per arriving EV. First, the simulator projects FCS waits for the arriving EV using the current FCS port schedules. Then it decides whether to deploy an idle MCV from the depot.

7.1 Deterministic Thresholds

Policy mode	Wait threshold	Max deployments per EV arrival
`peak_resilience`	10 min	2
`revenue`	18 min	1
Other	14 min	1

For deterministic operator control, candidate FCSs are sorted by projected wait in descending order. An MCV is deployed to a candidate FCS only if:

1. projected wait exceeds the threshold,
2. the per-cycle deployment limit is not reached,
3. fewer than 2 MCVs are already active or enroute for that FCS,
4. at least one MCV is idle at the depot,
5. the idle MCV has energy above its reserve threshold.

When deployed, MCV depot-to-FCS travel time is computed using Manhattan distance from $(10, 10)$ to the target FCS. The MCV becomes available at:

$$T_{mcv}^{available} = t_{now} + T_{depot,FCS}^{travel}.$$

7.2 LLM Operator Control

When operator LLM control is enabled, the LLM receives:

- policy mode,
- policy knobs,
- arriving EV context,
- projected FCS waits,
- FCS status,
- MCV status.

It returns JSON with:

```
{
  "deploy_fcs_ids": ["FCS_1"],
  "explanation": "brief reason"
}
```

Only existing FCS IDs are accepted. Invalid target IDs are ignored and counted as invalid LLM output in the extended evaluator.

8 EV Decision Logic

8.1 Deterministic Fixed Controller

The base deterministic controller builds all feasible options, scores them by EV strategy, sorts ascending by score, and chooses rank 1.

8.2 Time-Optimized EV

$$\text{score}_{e,r} = T_{e,r}^{total}.$$

The chosen option minimizes travel plus wait plus service time.

8.3 Cost-Optimized EV

$$\text{score}_{e,r} = C_{e,r}.$$

The chosen option minimizes monetary charging cost.

8.4 Greedy EV

For greedy EVs, first compute EV-side raw preference components:

$$a_t = 1 - \text{waiting\_tolerance}, \quad a_c = \text{price\_sensitivity}, \quad a_d = 1 - \text{detour\_tolerance}.$$

Normalize them:

$$w_t^{EV} = \frac{a_t}{a_t + a_c + a_d}, \quad w_c^{EV} = \frac{a_c}{a_t + a_c + a_d}, \quad w_d^{EV} = \frac{a_d}{a_t + a_c + a_d}.$$

Policy profiles are:

$$\mathbf{w}_{peak}^{policy} = (0.70, 0.10, 0.20),$$

$$\mathbf{w}_{revenue}^{policy} = (0.15, 0.70, 0.15).$$

The driver-preference emphasis β blends EV preference and operator policy:

$$\tilde{w}_j = \beta w_j^{EV} + (1 - \beta) w_j^{policy}, \quad j \in \{t, c, d\}.$$

The resulting $\tilde{w}_j$ values are normalized to sum to one.
For each option:

$$n_t = \text{norm}(T_{e,r}^{total}), \quad n_c = \text{norm}(C_{e,r}), \quad n_d = \text{norm}(d(e, r)).$$

The min-max normalization is:

$$\text{norm}(x) = \begin{cases} 0.5, & x_{max} = x_{min}, \\ \frac{x - x_{min}}{x_{max} - x_{min}}, & \text{otherwise.} \end{cases}$$

The MCV adjustment is:

$$A_{mcv} = 0$$

for FCS options. For MCV options:

$$A_{mcv} \leftarrow A_{mcv} + \max(0, 0.5 - \text{mcv\_acceptance}) \times 0.25.$$

If policy is `peak_resilience` and MCV acceptance is at least 0.5:

$$A_{mcv} \leftarrow A_{mcv} - 0.08.$$

If policy is `revenue`:

$$A_{mcv} \leftarrow A_{mcv} + 0.03.$$

Final greedy score:

$$\text{score}_{e,r} = \tilde{w}_t n_t + \tilde{w}_c n_c + \tilde{w}_d n_d + A_{mcv}.$$

Lower score is better.

9 Satisfaction Score

The satisfaction score is EV user satisfaction, not operator revenue. It is computed for a chosen option relative to the feasible option set available to that EV.

Let:

$$T^* = \min_r T_{e,r}^{total}, \quad C^* = \min_r C_{e,r}, \quad W^* = \min_r T_{e,r}^{wait}.$$

Utilities:

$$u_t = \min\left(1, \frac{T^*}{T_{e,chosen}^{total}}\right),$$

$$u_c = \min\left(1, \frac{C^*}{C_{e,chosen}}\right),$$

$$u_w = \exp\left(-\frac{\max(0, T_{e,chosen}^{wait} - W^*)}{10}\right),$$

$$u_r = \begin{cases} 1, & \text{reservation feasible,} \\ 0, & \text{otherwise.} \end{cases}$$

The final satisfaction score is:

$$S_e = 100(w_t u_t + w_c u_c + w_w u_w + w_r u_r).$$

Here w_t, w_c, w_w, and w_r are intent-specific weights for total time, cost, waiting time, and reliability:

Intent	w_t	w_c	w_w	w_r
`time_optimized`	0.55	0.15	0.20	0.10
`cost_optimized`	0.15	0.60	0.15	0.10
`greedy`	0.35	0.30	0.20	0.15

10 Regret Score

Decision regret is a normalized gap between the chosen deterministic strategy score and the best available strategy score:

$$R_e = 100\frac{\max(0, \text{score}_{chosen} - \text{score}_{best})}{\text{score}_{worst} - \text{score}_{best}}.$$

If all option scores are equal, regret is zero.

11 LLM Boundedness Contract

The LLM never creates a physical action directly. The simulator first computes valid options. The LLM only selects from those options or recommends deployment to existing FCS IDs.

11.1 EV LLM Payload

The EV LLM receives:

- policy mode,
- EV ID, model, location, state of charge, target state of charge,
- EV strategy and written situation,
- operator notes,
- ranked feasible options,
- per-option travel, wait, service, total time, cost, price,
- deterministic strategy score,
- estimated satisfaction if chosen,
- reservation availability.

It must return JSON:

```
{
  "chosen_option_id": "FCS_1",
  "reason": "brief reason",
  "confidence": 0.0
}
```

The code uses `temperature=0.1` and JSON response format.

11.2 Fallback Rules

In the normal simulator, if the EV LLM fails or returns an invalid option, the already-selected deterministic top-ranked option remains in force.

In the extended evaluator, if the EV LLM fails or returns an invalid option, the fallback is `deterministic_satisfaction`. The event is logged in `invalid_llm_output_count` and/or `fallback_count`.

12 Normal Evaluation

12.1 Runners

```
python run_deterministic.py
python run_llm.py
python run_compare.py
```

12.2 Normal Evaluation Outputs

- `ev_charging_agents/flow_log.txt`
- `ev_charging_agents/flow_logs/deterministic_vs_llm_comparison.txt`
- `ev_charging_agents/flow_logs/deterministic_vs_llm_comparison.csv`
- `ev_charging_agents/flow_logs/deterministic/`
- `ev_charging_agents/flow_logs/llm/`

12.3 Normal Evaluation Result

Metric	Deterministic	LLM-assisted
EV count	10	10
Average satisfaction	85.8/100	96.8/100
Average wait	12.47 min	2.47 min
Resource choices	10 FCS	10 FCS
Accepted EV LLM picks	N/A	10/10
Satisfaction delta	N/A	+11.0 points

13 Extended Evaluation

13.1 Runners

From the project root:

```
python run_extended_eval.py
```

From inside the `ev_charging_agents` folder:

```
python run_extended_eval.py
```

13.2 Outputs

- `ev_charging_agents/flow_logs/extended_eval/`

13.3 Controller Set

- `deterministic_fixed`: choose deterministic rank 1.
- `deterministic_satisfaction`: choose maximum satisfaction, tie-breaking by lower wait, lower cost, then lower total time.
- `llm_full`: operator LLM and EV LLM enabled.
- `llm_operator_only`: operator LLM enabled; EV choice uses deterministic satisfaction.
- `llm_ev_only`: deterministic operator; EV LLM enabled.

13.4 Stage 1 Demand Sweep

Operator policy remains `revenue`. Physical topology is unchanged. Interarrival is 60 seconds. EV counts:

$$\{10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30\}.$$

For each EV count, run:

$$\{\texttt{deterministic\_fixed}, \texttt{deterministic\_satisfaction}, \texttt{llm\_full}\}.$$

One sweep seed is used: 20260427.

13.5 Threshold Logic

$$\texttt{first\_count\_mcv\_selected} = \min\{N : \texttt{mcv\_selected\_count} > 0\}.$$

$$\texttt{first\_count\_mcv\_dispatch} = \min\{N : \texttt{mcv\_dispatch\_count} > 0\}.$$

$$\texttt{threshold\_scenario} = \begin{cases} \texttt{first selected scenario}, & \text{if it exists,} \\ \texttt{first dispatch scenario}, & \text{else if it exists,} \\ \texttt{null}, & \text{otherwise.} \end{cases}$$

Observed threshold:

Field	Value
first_count_mcv_selected	10
first_count_mcv_dispatch	10
threshold_scenario	stage1_ev10_ia60
strong_stress_scenario	stage1_ev30_ia60
fallback_burst_stage_needed	false
fallback_outage_stage_needed	false

13.6 Stage 2 Fallback

Stage 2 is implemented but was not needed in the completed run. If no hybrid activity appears by 30 EVs, the evaluator tries:

- burst scenarios with EV counts 20, 24, 28, 30 compressed into a roughly 10-minute window;
- if still needed, a mild outage where FCS_2 is unavailable until minute 12.

13.7 Stage 3 Representative Reruns

Representative scenarios:

- routine: 10 EVs, 60 s interarrival;
- hybrid threshold: discovered threshold scenario;
- strong stress: highest-load scenario with strongest hybrid activity, here 30 EVs, 60 s interarrival.

Representative seeds:

$$\{20260427, 20260428, 20260429\}.$$

Role ablations are added only to hybrid-threshold and strong-stress scenarios.

14 Extended Evaluation Outputs

All outputs are written under:

`ev_charging_agents/flow_logs/extended_eval/`

Output file	Meaning
`run_manifest.json`	Scenario names, controller names, seeds, EV counts, interarrival settings, burst/outage settings, and timestamp.
`sweep_results.csv`	One row per Stage 1/2 sweep run.
`threshold_summary.json`	First MCV-selected count, first MCV-dispatch count, selected threshold scenario, strong-stress scenario, and fallback flags.
`representative_results.csv`	One row per representative run and seed.
`aggregate_summary.csv`	Mean/std aggregation for representative scenario-controller pairs.
`per_ev_choices.csv`	One row per EV decision, including chosen resource, travel, wait, service, total time, cost, satisfaction, and decision source.
`compliance_audit.csv`	One row per LLM EV decision, including feasible option count, selected option ID, feasible-set check, invalid flag, fallback flag, and chosen resource type.
`key_examples.txt`	Concise examples where choices changed meaningfully.
`main_table.csv`	Compact table used for paper reporting.
`main_table.tex`	Auto-generated LaTeX table from main table rows.

15 Representative Results

Scenario	Controller	Sat.	Wait	Total	MCV sel.	Calls
Hybrid threshold	`deterministic_fixed`	85.03	12.52	44.42	0.00	0
Hybrid threshold	`deterministic_satisfaction`	94.87	0.69	43.49	2.67	0
Hybrid threshold	`llm_ev_only`	95.17	1.19	43.07	2.00	10
Hybrid threshold	`llm_full`	96.73	1.57	46.03	0.00	20
Hybrid threshold	`llm_operator_only`	97.77	1.19	47.47	0.00	10
Strong stress	`deterministic_fixed`	80.50	15.08	47.80	3.67	0
Strong stress	`deterministic_satisfaction`	94.30	0.50	43.48	13.00	0
Strong stress	`llm_ev_only`	93.40	1.57	41.87	13.67	30
Strong stress	`llm_full`	97.47	5.61	54.07	2.33	60
Strong stress	`llm_operator_only`	98.03	6.16	55.70	1.33	30

16 Stage 1 Sweep Summary

Across the 11 demand-sweep EV counts:

Controller	Runs	Mean sat.	Mean wait	MCV selected sum	LLM calls
`deterministic_fixed`	11	83.83	12.90	30	0
`deterministic_satisfaction`	11	94.77	0.34	72	0
`llm_full`	11	97.12	3.94	0	440

This summary shows why the strong comparator matters. The `deterministic_satisfaction` controller is not weak; it aggressively uses MCV choices to minimize EV dissatisfaction. The LLM-based controller achieves high satisfaction while preserving a more revenue-aware and less MCV-aggressive behavior.

17 Compliance and Call Budget

Across the full extended evaluation:

Part	Runs	Total calls	Operator calls	EV calls
Sweep	33	440	220	220
Representative	39	540	270	270
Total	72	980	490	490

Compliance audit:

Controller	Audit rows	Feasible selections	Invalid outputs	Fallbacks
`llm_full`	370	370	0	0
`llm_ev_only`	120	120	0	0
Total	490	490	0	0

18 Algorithmic Flow

Normal or Extended Single Run

1. Initialize FCS resources, MCV fleet, operator policy, and EV request list.
2. For each EV in arrival order:
   (a) Advance MCV statuses from enroute to deployed if travel time has elapsed.
   (b) Project FCS waits for this EV.
   (c) Run operator control. This may deploy an MCV.
   (d) Build feasible options: all FCS options plus deployed/assigned MCV options that have enough energy.
   (e) Score options according to EV strategy.
   (f) Choose an option using the active controller.
   (g) Reserve the resource by updating the port heap and, for MCVs, onboard energy.
   (h) Compute satisfaction and regret.
   (i) Write per-EV log and append master summary.
3. Write run summary.

Extended Evaluation

1. Run Stage 1 EV-count sweep.
2. Compute first MCV-selected and first MCV-dispatch thresholds.
3. Run Stage 2 fallback only if no hybrid activity appears.
4. Select routine, hybrid-threshold, and strong-stress representative scenarios.
5. Rerun representative scenarios using three seeds.
6. Add role ablations for threshold and stress scenarios.
7. Write manifest, sweep, threshold, representative, aggregate, per-EV, compliance, key-example, and table files.

19 Interpretation Rules

- `wait_min` is queueing wait after travel, not total time.
- `total_time_min` is travel plus wait plus charging service.
- `mcv_dispatch_count` counts operator deployment actions.
- `mcv_selected_count` counts EV decisions that actually choose MCV service.
- A dispatched MCV may not be selected immediately if travel time, pricing, wait, or policy tradeoffs make FCS options more attractive.
- Satisfaction is EV-side user satisfaction, not operator profit.
- High satisfaction with higher wait can occur when the chosen option is still best for the EV's weighted intent, cost, and reliability tradeoff.
- The LLM result should not be interpreted as unconstrained autonomy. The LLM is bounded by feasible simulator-generated choices.

20 Research Claim Supported by the Experiment

The experiment supports the following bounded claim:

> In a hybrid FCS–MCV EV charging simulator, an LLM-assisted operator/EV coordination layer can operate over deterministic feasible options, preserve an auditable feasible-choice contract, and reshape the satisfaction-delay-resource tradeoff relative to fixed deterministic control and a strong satisfaction-aware non-LLM comparator.

It does not claim that LLMs replace optimization, MARL, power-electronics control, or physical safety systems. The intended role is tactical, event-triggered, explainable coordination over an existing charging engine.